

time. Therefore, many Lispers prefer to use functional techniques over object-oriented techniques, though the two can often be used together—with some care. Nonetheless, there are still many domains in which object-oriented techniques are invaluable, such as in user interface programming or simulation programming.

On the other hand, James Hague, in his assessment of functional programming argues that, “100 per cent pure functional programming doesn’t work. Even 98 per cent pure functional programming doesn’t work. But if the slider between functional purity and 1980s BASIC-style imperative messiness is kicked down a few notches—say to 85 per cent—then it really does work. You get all the advantages of functional programming, but without the extreme mental effort and un-maintainability that increases as you get closer and closer to perfectly pure.”

CLOS

If OO is what gets you going, Common Lisp offers the most sophisticated object-oriented programming framework of any major programming language. It’s called Common Lisp Object System (CLOS). It is customisable at a fundamental level, using the Meta-Object Protocol (MOP). It has been claimed that there’s really nothing like it anywhere else in programming. It lets you control incredibly complex software without losing control over the code.

Let the tears of joy flow...

Object-oriented programming in Common Lisp

What is CLOS?

(a) *It is a layered system designed for flexibility.*

One of the design goals of CLOS is to provide a set of layers that separate different programming language concerns from one another. The first level of the Object System provides a programmatic interface to object-oriented programming. This level is designed to meet the needs of most serious users, and to provide a syntax that is crisp and understandable. The second level provides a functional interface into the heart of the Object System. This level is intended for programmers who are writing very complex software or a programming environment. The first level is written in terms of this second level.

The third level provides the tools for programmers who are writing their own object-oriented language. It allows access to the primitive objects and operators of the Object System. It is this level at which the implementation of the Object System itself is based. The layered design of CLOS is founded on the meta-object protocol, a protocol that is used to define the characteristics of an object-oriented system. Using the meta-object protocol, other functional or

programmatic interfaces to the Object System, as well as other object systems, can be written.

(b) *It is based on the concept of generic functions rather than on message-passing.*

This choice is made for two reasons: 1) there are some problems with message-passing in operations of more than one argument; 2) the concept of generic functions is a generalisation of the concept of ordinary Lisp functions.

A key concept in object-oriented systems is that given an operation and a tuple of objects on which to apply the operation, the code that is most appropriate to perform the operation is selected, based on the classes of the objects. In most message-passing systems, operations are essentially properties of classes, and this selection is made by packaging a message that specifies the operation and the objects to which it applies, before sending that message to a suitable object. That object then takes responsibility for selecting the appropriate piece of code. These pieces of code are called methods.

(c) *It is a multiple inheritance system.*

Another key concept in object-oriented programming is the definition of structure and behaviour on the basis of the class of an object. Classes thus impose a type system—the code that is used to execute operations on objects depends on the classes of the objects. The sub-class mechanism allows classes to be defined that share the structure and the behaviour of other classes. This sub-classing is a tool for modularisation of programs.

(d) *It provides a powerful method combination facility.*

Method combination is used to define how the methods that are applicable to a set of arguments can be combined to provide the values of a generic function. In many object-oriented systems, the most specific applicable method is invoked, and that method may invoke other, less specific methods. When this happens, there is often a combination strategy at work, but that strategy is distributed throughout the methods as local control structure. Method combination brings the notion of a combination strategy to the surface, and provides a mechanism for expressing that strategy.

(e) *The primary entities of the system are all first-class objects.*

In the Common Lisp Object System, generic functions and classes are first-class objects with no intrinsic names. It is possible and useful to create and manipulate anonymous generic functions and classes. The concept of ‘first-class’ is important in Lisp-like languages. A first-class object is one that can be explicitly made and manipulated; it can be stored in any location that can hold general objects.

What CLOS is *not*

It does not make for a great pick-up conversation at the bar. I tried. It did not work!